PYTHON_ BY

ABDUL_REHMAN_AZAM_ _ _ _ _ _ _ _ _ _

Python programming

Data Structure

There are four types of data structure as given below:

1. List
2. Tuple
3. Sets
4. Dictionaries

➢ Now explain these types

1. List :-

List is a data structure to store multiple values in a single variable. List is mutable which means that we can modify it after making list. We can store different data type in a list enclosed in square brackets [ ] and separated by commas.

Example:

```
students = ["Ahmed", "Abdul Rehman", "Rayan"]
print(students)
```

📘 **Output: ["Ahmed", "Abdul Rehman", "Rayan"]**

We can also access ,modify, and perform different operations

🔶 **Accessing List Items:**

Use indexes (starting from 0):

```
fruits = ["Mango", "Apple", "Banana"]
print(fruits[1])     #output :Apple
```

Modifying List:

```
fruits[0] = "Orange"
fruits.append("Pineapple")
print(fruits)
```

Output: ['Orange', 'Apple', 'Banana', 'Pineapple']

Common List Methods:

Method	Function
append()	Adds an item to the end
remove()	Removes an item
sort()	Sorts the list
reverse()	**Reverses the list order**

➢ More operations in list (slicing + concatenation)

Slicing:

```
List2 = [1,2,3,4,5,6,7,8,9,10,11]
(list2[1:11:2])     # first step is starting index
 print(list2[1:11:3]) # second step is ending index
# third step is jump
```

Concatenation :

```
List1= [1,2,3,4]
List2= [5,6,7]
concatenate=List1+List2
```

```
Print(concatenate)  # output :[1,2,3,4,5,6,7]
```

2. Tuple :-

Tuple have the same function as the list but the key difference tuples are immutable means unchangeable after creating it. We can also store different data type in a list enclosed in parenthesses () and separated by commas.

Example:

```
Friends = ("Ahmad", "Rayan", "Ali")
Print (Friends)   # output :  ("Ahmad", "Rayan", "Ali")
```

➢ Some operations in tuple :

Here are some operations in tuple

- **Slicing:**

```
num = (1,2,3,4,5,6)
print (num [2:5]) # output: (3,4,5)
print (num [-1]) # output: (6)
```

- **finding the elements in tuple:**

```
number=(12,23,4,68,45)
if 45 in tuple:
  print("45 is present in number at index", number.index(45))
else:
  print("45 is not present in tuple")
```

output:

```
45 is present in number at index , 4
```

- **More operations :-**

```
Tuple = (2,4,3,8,5,4,6,3,6,634,45,0)
```

```
print (len(Tuple))          # it shows the length of tuple
print (Tuple.count(6))      # it counts the number that how many time it repeats
print (Tuple.index(634))    # it shows the index value of number
print ("3 at index after 4th index :",Tuple.index(3,4,8))
# -> first point is value to be searched
# -> second point is starting index
# -> third point is ending index
```

3. Sets:-

Sets are used to store multiple items in a single variable. A set is a collection which is *unordered*, *unchangeable**, and *unindexed*. Set *items* are unchangeable, but you can remove items and add new items. It can store all type of data. Sets are close in curly braces {}.

Example:

```
info = {"author", 10, 34.4, True, 10}
print(info)
```

Properties:-

1) Unordered

Unordered means that the items in a set do not have a defined order.

2) Unchangeable

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

Once a set is created, you cannot change its items, but you can remove items and add new items.

3) Duplicates Not Allowed

Sets cannot have two items with the same value.

Example:

```
num = {1,2,3,4,56,2,7,2}
print(num)
# output  : {1, 2, 3, 4, 7, 56}
```

➢ SETS METHODS AND OPERATIONS

1. **Union and update:**

   It join the two sets.

   Example:

```
# union inn sets join two sets
countries1={"Pakistan","India","China"}
countries2={"USA","UK","China"}
print(countries1.union(countries2))
# update method also joins two sets
countries1.update(countries2)
print(countries1)
```

2. **Intersection and update:**

   It also join the two sets with common numbers in it.

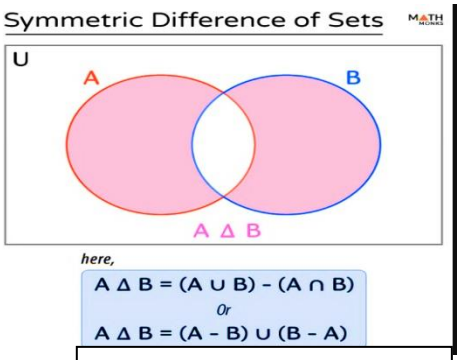
   Example:

```
# intersection in sets gives common values
cities1 = {"Karachi", "Lahore", " Multan "}
cities2 = {"Karachi", "Islamabad"," Multan "}
print(cities1.intersection(cities2))
# intersection_update method also gives common values
cities1.intersection_update(cities2)
```

print(cities1)

3. **Symmetric difference:**

Symmertic difference gives values that are not common in both sets.



Symmetric difference

Example:

```
friends1={"Ahmad","Rayan","Rafy","Faizan"}
friends2={"Asad","Hussnain","Rayan","Rafy"}
friends=friends1.symmetric_difference(friends2)
friends1.symmetric_difference_update(friends2)
print(friends)
print(friends1)
```

4. **Difference:**

difference gives values that are not common in both sets but only from first set
Formula is A-B
Example:

```
A={"A","B","C","D"}
B={"A","E","F","G"}
c=A.difference(B)
B.difference_update(A)
print(c)
print(B)
```

5. **Is super set:**

Issuperset method returns true if all values of set B are present in set A.

Example:

```
value1={1,2,3,4,5}
```

value2={1,2,3,4}

print(value1.issuperset(value2))

6. **Is sub set:**

issubset method returns true if all values of set A are present in set B.

Example:

num1={1,2,3,4,5,6}
num2={1,2,3,4,5}
print(num2.issubset(num1))

7. **Add item:**

You can add item is set through this syntax.

Example:

cousins ={"Huzaifa","Hanzala","Adil","Moiz"}
cousins.add("Hamza")
print(cousins)

8. **Remove | discard item:**

Remove function remove the item in the set and discard also has same function but the key difference is discard doesn't generate error if item is not present in a set.

Example:

cousins ={"Huzaifa","Hanzala","Adil","Moiz"}

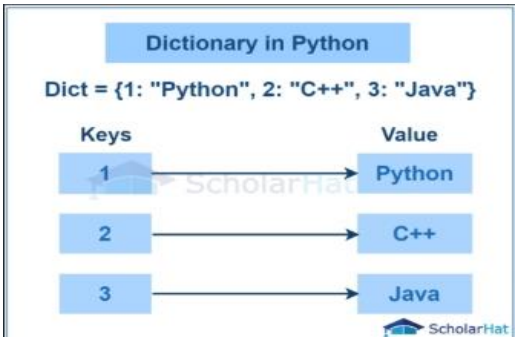
cousins.remove("Moiz")

print(cousins)
cousins.discard("ali")
print(cousins)
# pop method
item=cousins.pop()
print("The remove item from the set is ",item)
print(cousins)

4. Dictionaries:

Dictionaries are used to store data value in key:value pair.It is a collection which is *orders,mutable and do not allow duplicates .

Dictionaries are written with curly brackets, and have keys and values:



Example:

```
dic = {
"Name" :"Abdul Rehman",
"Age" : 17,
"city" : "Chishtian",
"ph_num": "0307 4809035"
 }
 print(dic)
```

Properties:

1. It is mutable which means that changeable.
2. It is in ordered.
3. It doesn't allow duplicates.

Operatios in dictionaries:

- **Accessing items:**

```
dict ={
   "name": "Dictionary Method",
```

```
    "age": 1,
    "city": "Internet",
    "type": "Data Structure"
}
print(dic.items())
```

- **Accessing keys:**

```
dict ={
    "name": "Dictionary Method",
    "age": 1,
    "city": "Internet",
    "type": "Data Structure"
}
print(dic.keys())
```

- **Accessing values:**

```
print(dic.values())
```

More methods in dictionaries:

- **Update method:**

update method is used to add the elements of one dictionary to another dictionary

Example:

```
std1={
    "Mana": 89,
    "Ahmad": 78,
    "Faizan": 91,
    "Rayan": 63
    }
    std2 ={
    "Ayan":88,
    "Dawood":95,
    "umair":79
    }
std1.update(std2)
print(std1)
```

- **Clear method :**

clear method is used to remove all the elements from the dictionary

Example:

```
std1={
        "Mana": 89,
        "Ahmad": 78,
        "Faizan": 91,
        "Rayan": 63
        }
        std2 ={
        "Ayan":88,
        "Dawood":95,
        "umair":79
        }
std2.clear()

print(sts2)
```

- **pop and del method:**

pop method removes the item with the specified key name . popitem method removes the last inserted item from the dictionary.

Example:

```
std1={
        "Mana": 89,
        "Ahmad": 78,
        "Faizan": 91,
        "Rayan": 63
        }
        std2 ={
        "Ayan":88,
        "Dawood":95,
        "umair":79
        }
std1.pop("Mana")
```

print(std1)

std1.popitem()

print(std1)

del :

del method removes the dictionary completely

above the same example :

del std2

print(std2)

 # it will give error because std2 is deleted

 # but you can also use del method to remove a specific item from the dictionary

del std1["Dawood"]

print(std1)

THE END